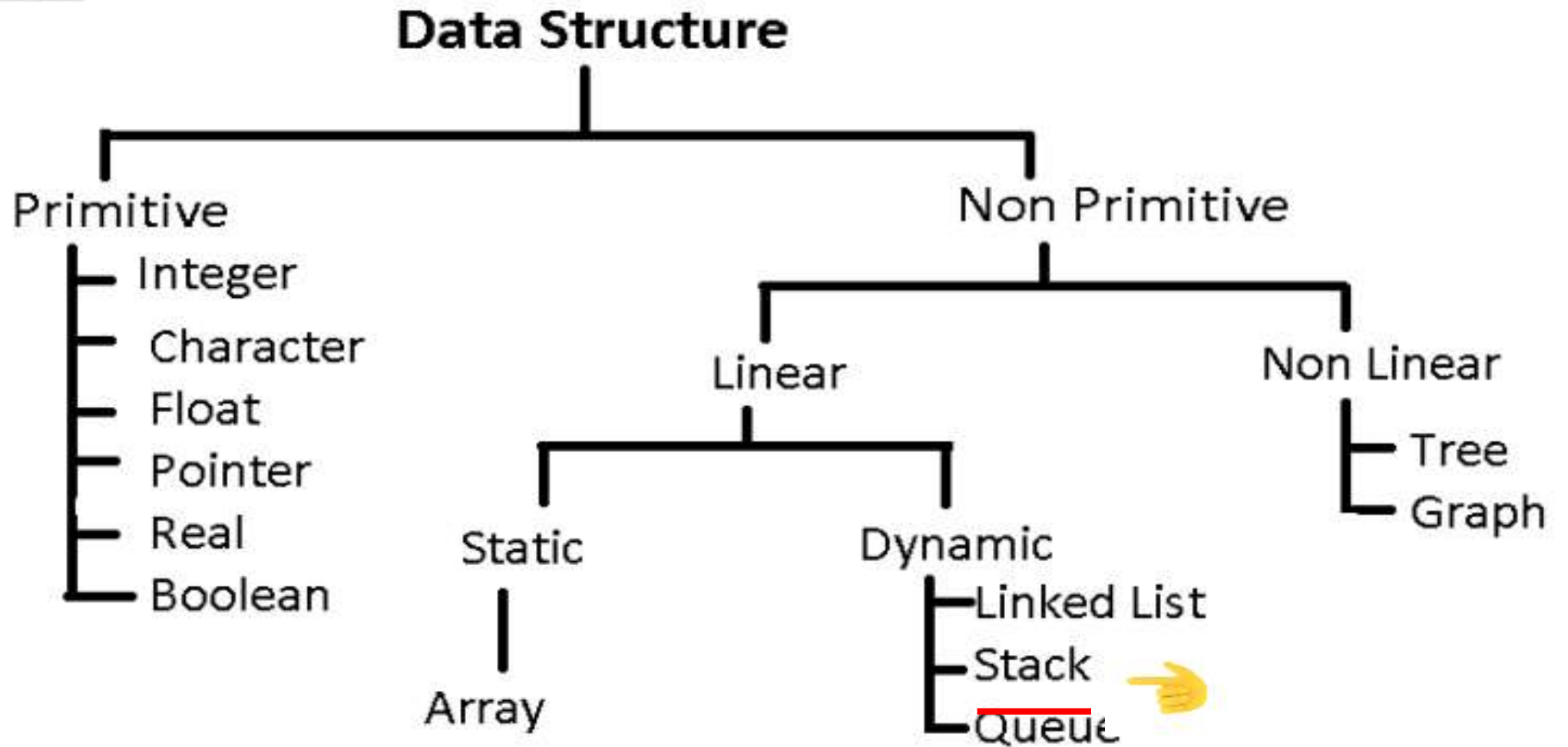


STACKS

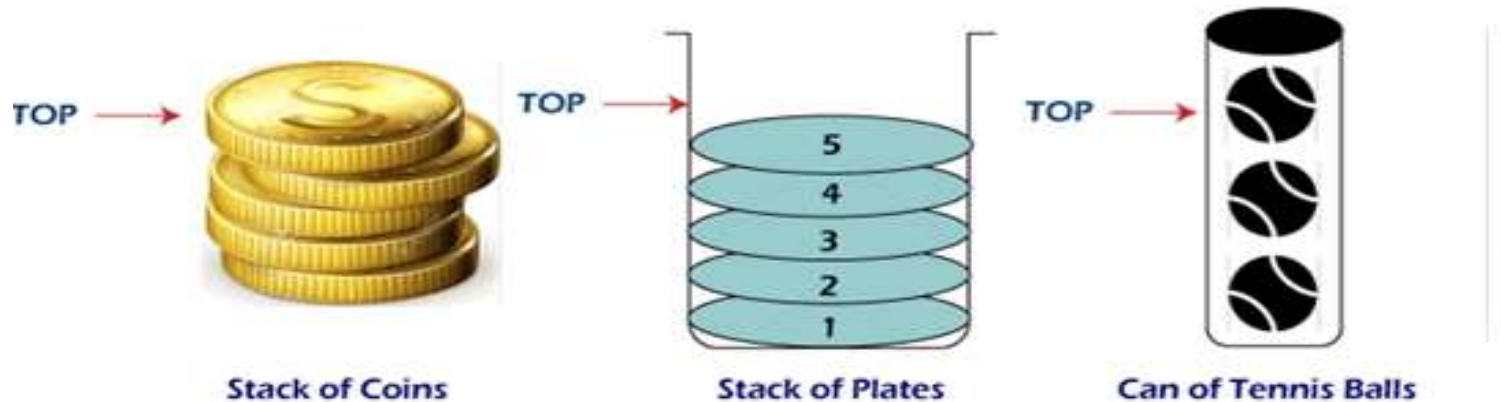
Dr. Jyothi Pillai
Professor
Bhilai Institute of Technology Durg(C.G.)

STACKS



Stacks

- ▶ A stack, a last-in-first-out (LIFO) system, is a list in which items may be inserted to or removed from only at one end called the top of the stack.
- ▶ A stack may be in our daily life, for example a stack of dishes.
 - Dish may be added or removed only from the top of the Stack.
 - The plate which is at the top is the first one to be removed, i.e. the plate which has been placed at the bottommost position remains in the stack for the longest period of time.

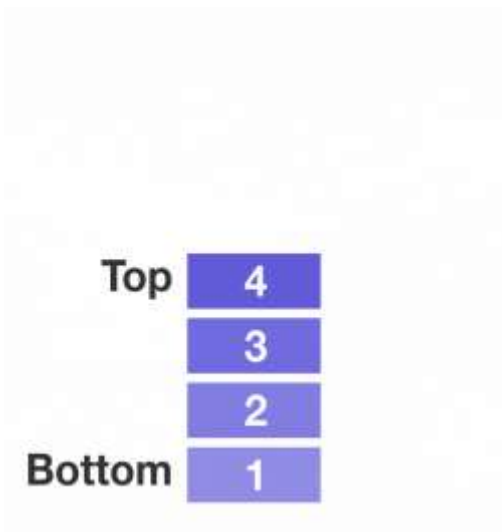
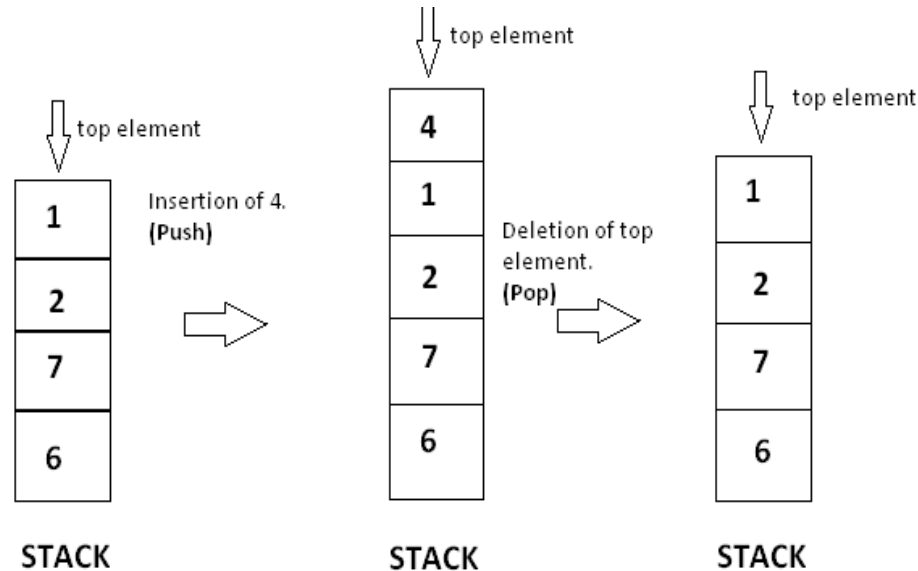
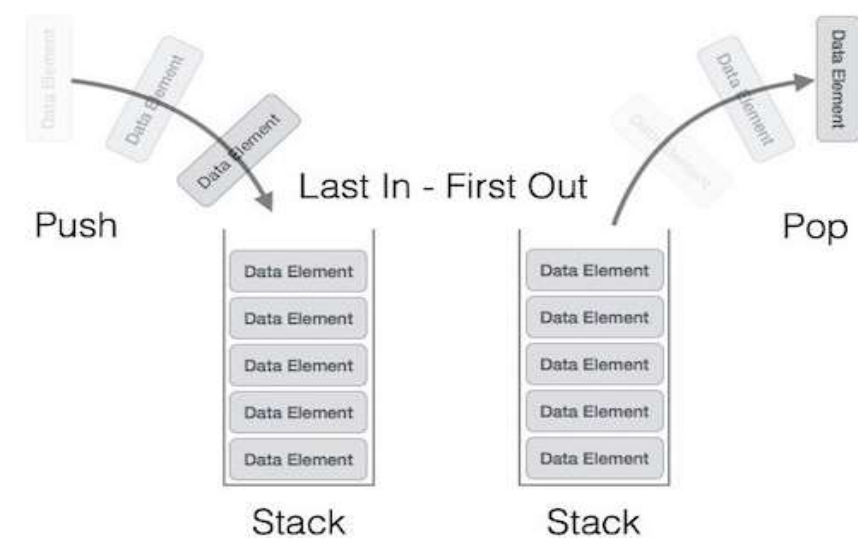


Main stack operations

A stack is used for the following two primary operations, **push(Insertion)** and **pop(Deletion)** –

- **push()** – Pushing / Inserting an element on the stack.
- **pop()** – Removing / Deleting an element from the stack.

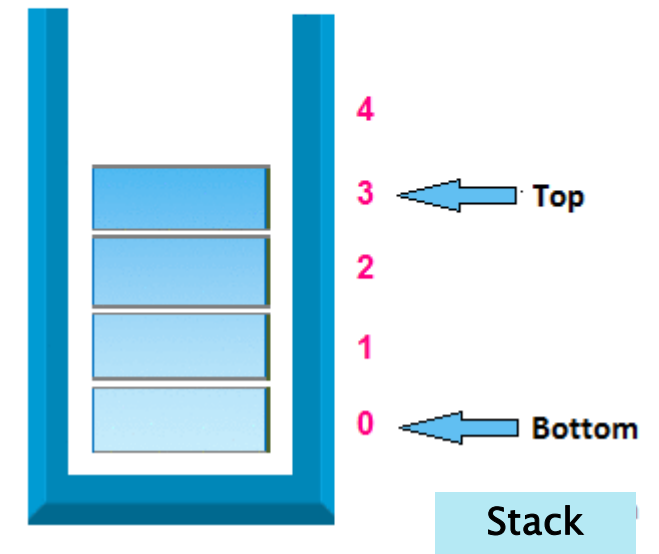
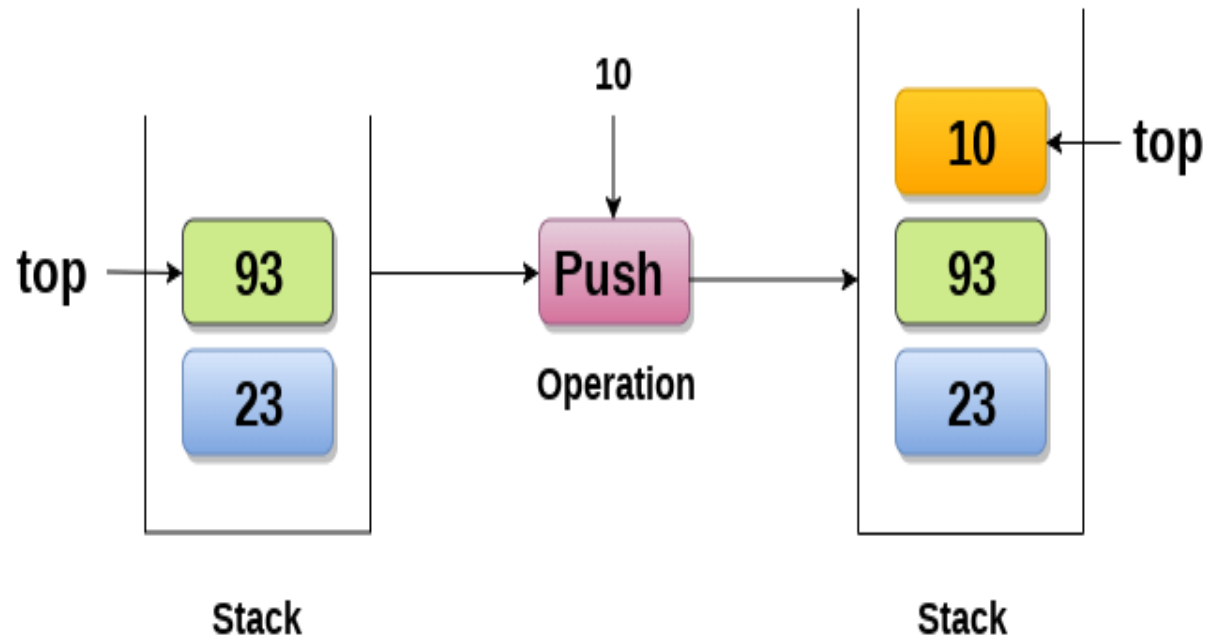
Stack restriction - Elements can be inserted or deleted only from one end i.e. **top** of the stack.



Main stack operations

❖ PUSH

- ❖ Process of putting a new data element onto stack is known as a Push.
- ❖ Pushing (storing) an element onto the top of the stack (insert an item).



PUSH

- ▶ Push operation involves a series of steps –
 - ❖ **Step 1** – Checks if the stack is full.
 - ❖ **Step 2** – If the stack is full, produces an error and exit.
 - ❖ **Step 3** – If the stack is not full, increments **top** to point next empty space.
 - ❖ **Step 4** – Adds data element to the stack location, where top is pointing.
 - ❖ **Step 5** – Returns success.

Algorithm

```
begin procedure push: stack, data
    if stack is full
        return null
    endif
    top ← top + 1
    stack[top] ← data
end procedure
```

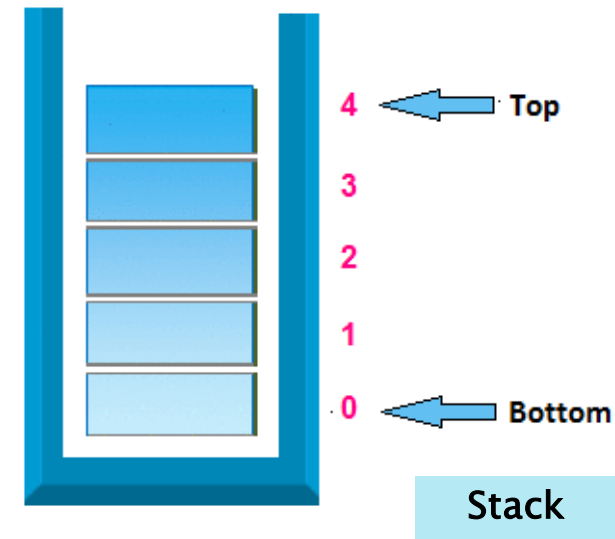
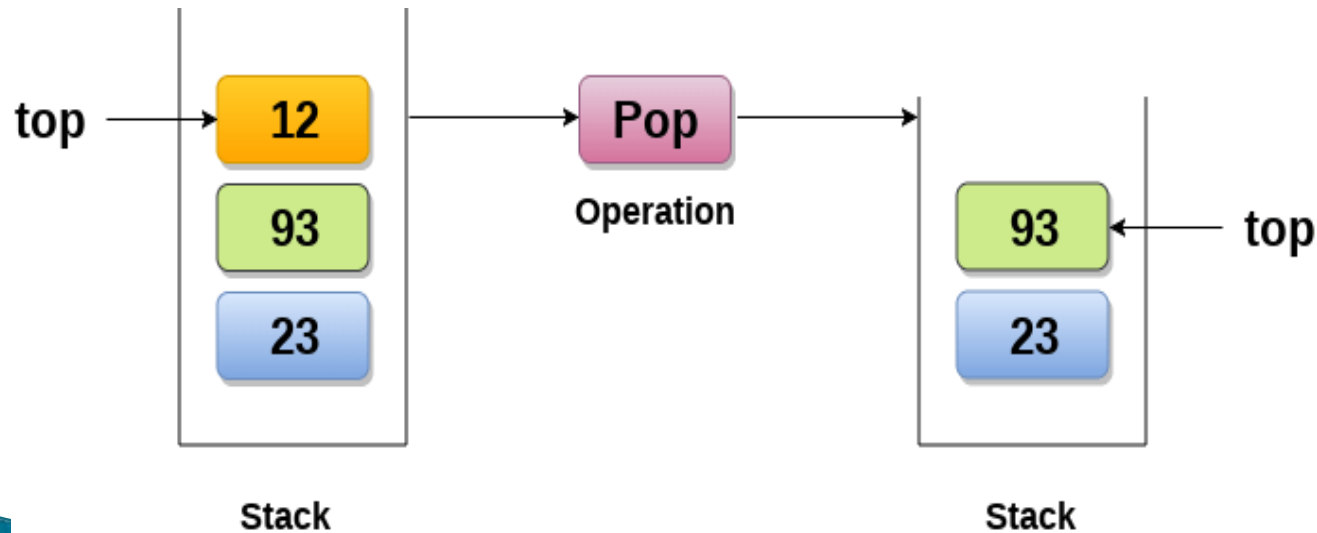
C program

```
void push(int data)
{ if(isFull())
    {printf("Overflow");}
  else
    {top = top + 1;
      stack[top] = data;}
}
```

Main stack operations

❖ POP

- ❖ Accessing the content while removing it from the stack, is known as a Pop.
- ❖ Popping (Removing/accessing) an element off the top of the stack (delete an item).



POP

- ▶ Pop operation involves the following steps –
 - ❖ **Step 1** – Checks if the stack is empty.
 - ❖ **Step 2** – If the stack is empty, produces an error and exit.
 - ❖ **Step 3** – If the stack is not empty, accesses the data element at which top is pointing.
 - ❖ **Step 4** – Decreases the value of top by 1.
 - ❖ **Step 5** – Returns success.

Algorithm

```
begin procedure pop: stack
  if stack is empty
    return null
  endif
  data ← stack[top]
  top ← top - 1
  return data
end procedure
```

C program

```
int pop(int data)
{
  if(!isempty())
  {
    data = stack[top];
    top = top - 1;
    return data;
  }
  else
  {
    printf("Could not retrieve
    data, Stack is empty.\n");
  }
}
```

Performance

- ▶ Let n be the number of elements in the stack.
- ▶ The complexities of stack operations are:-

Representation can be given as:

Time Complexity of Push()	$O(1)$
Time Complexity of Pop()	$O(1)$
Time Complexity of Display()	$O(n)$
Time Complexity of IsEmptyStack()	$O(1)$
Time Complexity of IsFullStackf)	$O(1)$

Auxiliary stack operations

- ❖ **Peek**
 - ❖ get the top data element of the stack, without removing it.
- ❖ **isFull**
 - ❖ check if stack is full.
- ❖ **isEmpty**
 - ❖ check if stack is empty.

peek

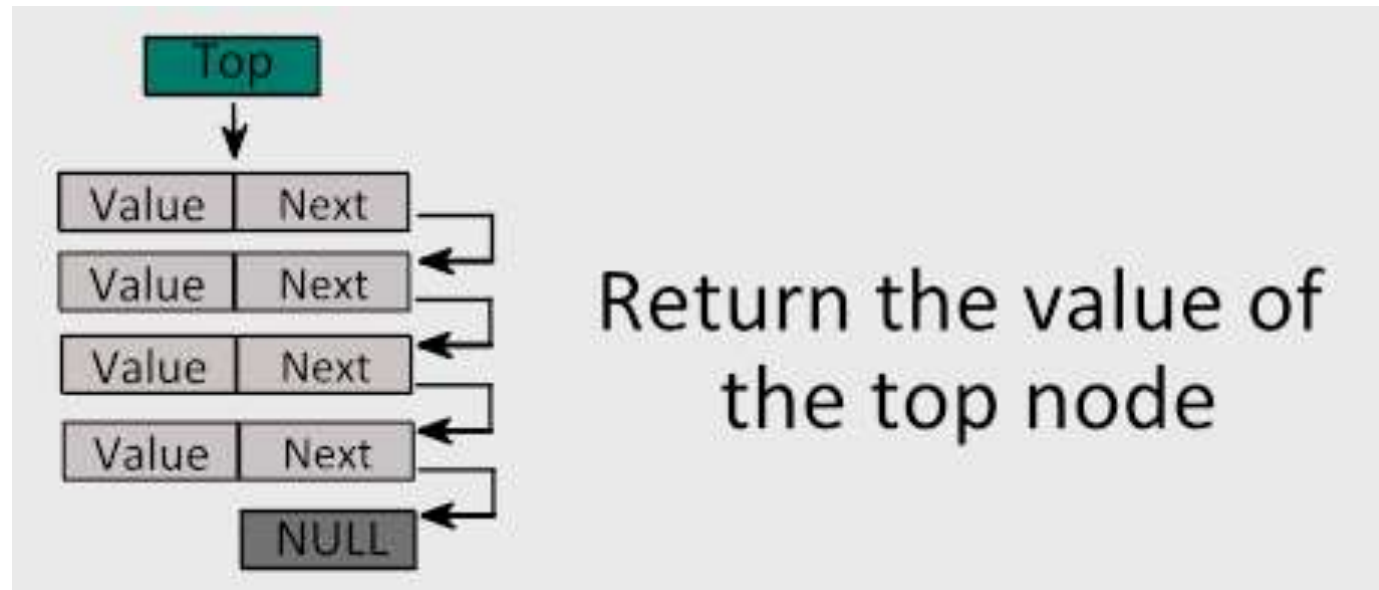
- ❖ To get the top data element of the stack, without removing it.

Algorithm

```
begin procedure peek  
    return stack[top]  
end procedure
```

C program

```
int peek()  
{ return stack[top];  
}
```



isfull

- ❖ To check if stack is full

Algorithm

```
begin procedure isfull  
  
    if top equals to MAXSIZE  
        return true  
    else  
        return false  
    endif  
  
end procedure
```

C program

```
bool isfull()  
{  
    if(top == MAXSIZE)  
        return true;  
    else  
        return false;  
}
```

isempty

- ❖ To check if stack is empty

Algorithm

```
begin procedure isempty  
  
    if top less than 1  
        return true  
    else  
        return false  
    endif  
  
end procedure
```

C program

```
bool isempty()  
{  
    if(top == -1)  
        return true;  
    else  
        return false;  
}
```

Applications

Following are some of the applications in which stacks play an important role:–

❖ Direct applications

- ❖ Balancing of symbols
- ❖ Infix-to-postfix conversion
- ❖ Evaluation of postfix expression
- ❖ Implementing function calls (including recursion)
- ❖ Finding of spans (finding spans in stock markets, refer to Problems section)
- ❖ Page-visited history in a Web browser [Back Buttons]
- ❖ Undo sequence in a text editor
- ❖ Matching Tags in HTML and XML

Applications

❖ Indirect applications

- ❖ Auxiliary data structure for other algorithms (Example: Tree traversal algorithms)
- ❖ Component of other data structures (Example: Simulating queues)

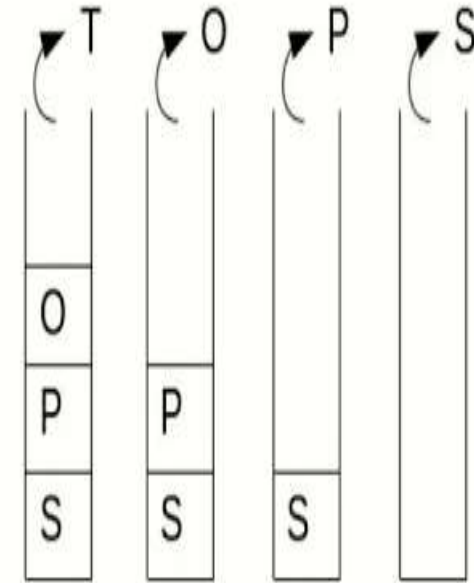
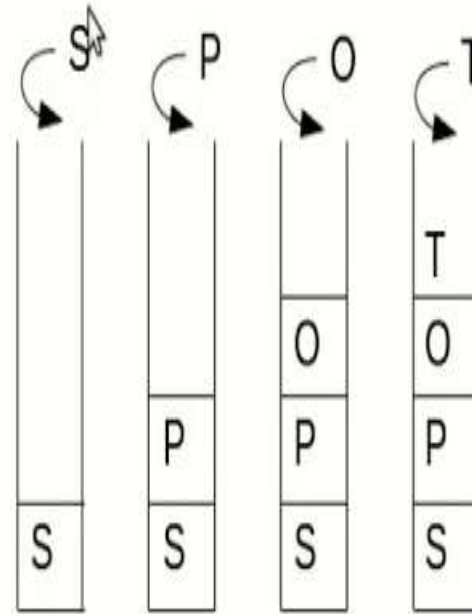
Stack Application– Reversing a string

Algorithm to reverse a string

1. Given the string and a stack
2. While there is not end of string, do the following:–
 3. Read a character form the string
 4. Push it on the stack
5. End while
6. Re-initialize string position
7. While stack is not Empty, do the following:–
 8. Pop a character from the stack
 9. Insert the character popped into next position in string
10. End While

String Reversing

```
void main( )  
{  
    char str[ ] = "SPOT";  
    cout<<" Original String is "<<str;  
    reverse(str);  
    cout<<" Reversed String is "<<str;  
    getch();  
}
```



Reversed String: TOPS

Expression

$\langle \text{operand} \rangle \langle \text{operator} \rangle \langle \text{operand} \rangle$

Operands are basic objects on which operation is performed.

Operators are used to indicate the operation performed on operands.

- $2+3$
 - $A-b$
 - $(P*2)$
- } Common expressions

Expression Notations

Infix, Prefix & Postfix

Infix

- An expression in which the operator is in between its two operands.
 - ▶ **A+B**
 - ▶ **<operand> <operator> <operand>**
 - $(2+3) *$
 - $(P+Q)*(R+S)$

Infix

Order of precedence BODMAS

1. $()\{\}\square$
2. $\wedge$ $R \rightarrow L$
3. $*/$ $L \rightarrow R$
4. $+ -$ $L \rightarrow R$

Prefix (Polish notation)

- ❖ In prefix notation the operator precedes the two operands.
i.e. the operator is written before the operands.

<operator> <operand> <operand>

- ❖ Infix $\rightarrow$ prefix

▶ $2+3$	$\rightarrow$	$+23$
▶ $p-q$	$\rightarrow$	$-pq$
▶ $A*B+C$	$\rightarrow$	$+*ABC$

Postfix (Reverse Polish Notation)

- ❖ In postfix notation the operators are written after the operands so it is called the postfix notation (post means after).

<operand> <operand> <operator>

❖	INFIX	→	PREFIX	→	POSTFIX
▶	2+3	→	+23	→	23+
▶	p-q	→	-pq	→	pq-
▶	A*b+c	→	+*abc	→	abc*+

Application – Infix to Postfix Using STACK

1. Scan the input string (infix notation) from left to right.
2. If the next symbol scanned is an operand, it may be immediately appended to the postfix string.
3. If the next symbol is an operator,
 - i. Pop and append to the postfix string every operator on the stack that
 - a. is above the most recently scanned left parenthesis
 - b. has precedence higher than or is a operator of equal precedence to that of the new operator.
 - ii. Push the new operator onto the stack.

Application – Infix to Postfix Using STACK

4. When a left parenthesis is seen, it must be pushed onto the stack.
5. When a right parenthesis is seen, all operators down to the most recently scanned left parenthesis must be popped and appended to the postfix string. Furthermore, this pair of parentheses must be discarded.
6. When the infix string is completely scanned, the stack may still contain some operators.
7. All remaining operators should be popped and appended to the postfix string.

Input expression: $A + B * C / D - E$

<i>Next Symbol</i>	<i>Postfix String</i>	<i>Stack</i>	<i>Rule</i>
A	A		2
+	A	+	3
B	A B	+	2
*	A B	+ *	3
C	A B C	+ *	2
/	A B C *	+ /	3
D	A B C * D	+ /	2
-	A B C * D / +	-	3
E	A B C * D / + E	-	2
	A B C * D / + E -		6

Input expression: $(A + B * (C - D)) / E$.

<i>Next Symbol</i>	<i>Postfix String</i>	<i>Stack</i>	<i>Rule</i>
(		(	4
A	A	(	2
+	A	(+	3
B	A B	(+	2
*	A B	(+ *	3
(	A B	(+ * (	4
C	A B C	(+ * (	2
-	A B C	(+ * (-	3
D	A B C D	(+ * (-	2
)	A B C D -	(+ *	5
)	A B C D - * +		5
/	A B C D - * +	/	3
E	A B C D - * + E	/	2
	A B C D - * + E /		6

Evaluation of POSTFIX Expression

1. While reading the expression from left to right, if the element is an operand, Push it into the stack.
2. If the element is an operator, Pop the two operands from the stack and then evaluate it.
3. Push back the result of the evaluation.
4. Repeat it till the end of the expression.

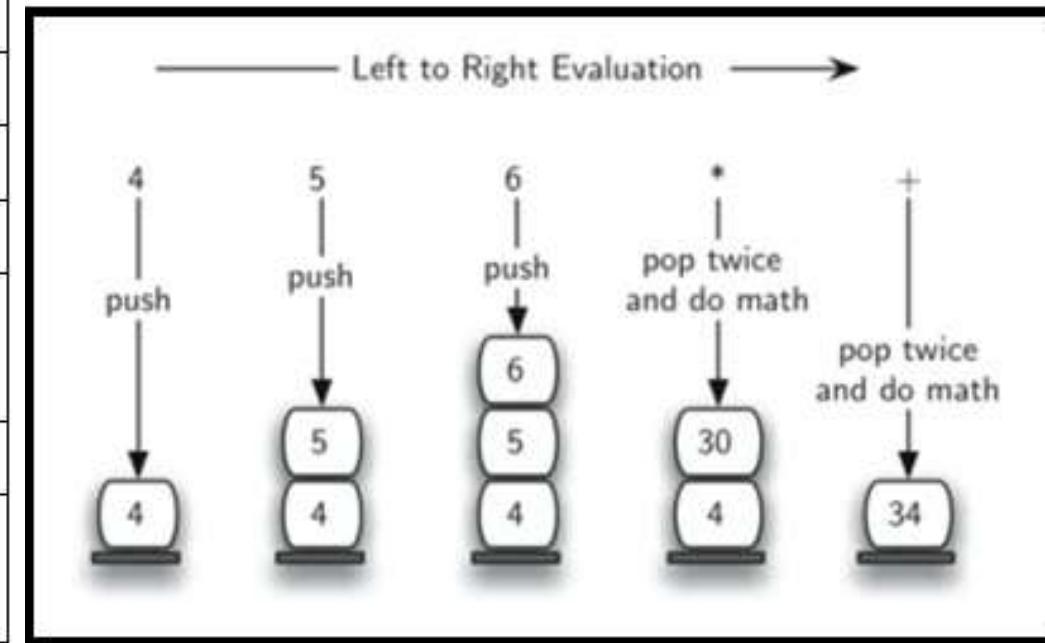
Algorithm

- 1) Add **)** to postfix expression.
- 2) Read postfix expression Left to Right until **)** encountered
- 3) If operand is encountered, push it onto Stack
[End If]
- 4) If an operator is encountered, Pop two elements
 - i) **A** -> **Top** element
 - ii) **B**-> **Next to Top** element
 - iii) Evaluate **B** operator **A**
 - iv) Push **B** operator **A** onto Stack
- 5) Set **result = pop**
- 6) END

Evaluation of POSTFIX Expression

► Expression: $456*+$

Step	Input Symbol	Operation	Stack	Calculation
1.	4	Push	4	
2.	5	Push	4,5	
3.	6	Push	4,5,6	
4.	*	Pop(2 elements) & Evaluate	4	$5*6=30$
5.		Push result(30)	4,30	
6.	+	Pop(2 elements) & Evaluate	Empty	$4+30=34$
7.		Push result(34)	34	
8.		No-more elements(pop)	Empty	34(Result)



INFIX TO PREFIX

The only change from postfix conversion is that the operator is placed before operand.

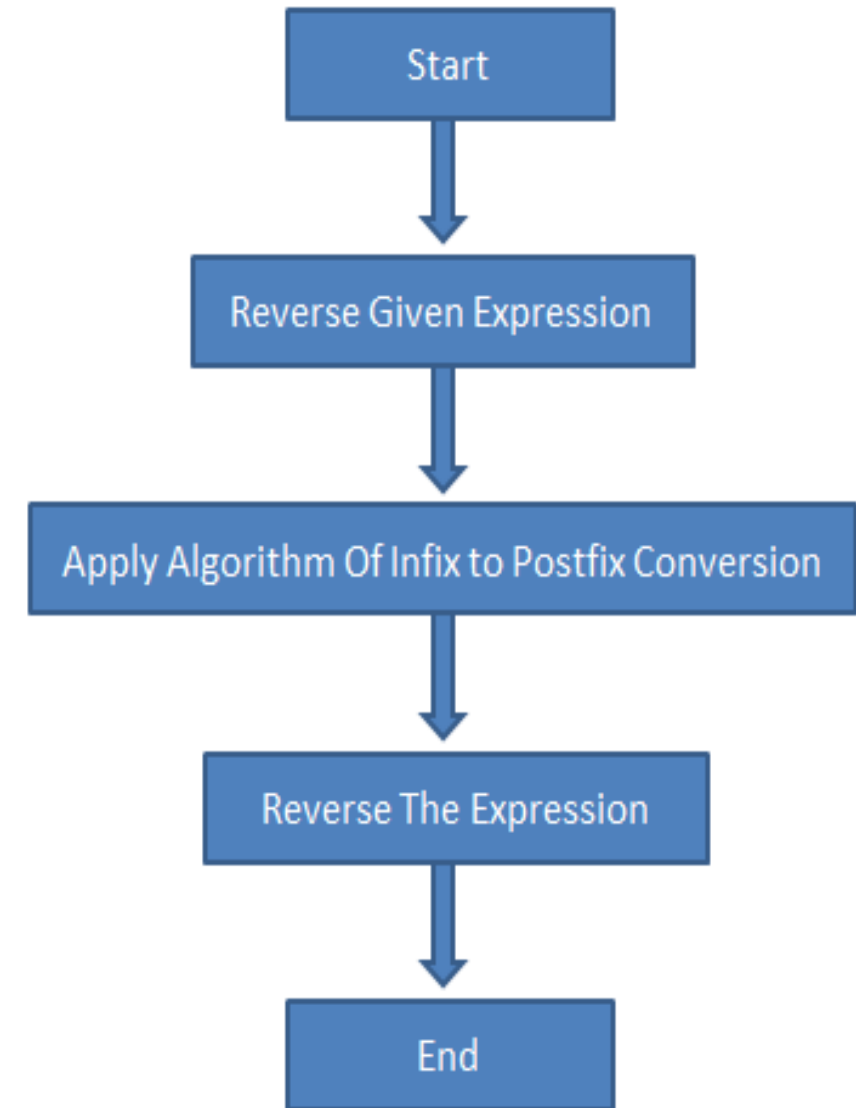
- ▶ $A+B \rightarrow +AB$
- ▶ $A+B-C \rightarrow +AB-C \rightarrow +-ABC$
- ▶ $(A+B)*(C-D) \rightarrow (+AB)*(-CD) \rightarrow *+AB-CD$
- ▶ $A^B*C-D+E/F/(G+H) \rightarrow ?$
- ▶ $((A+B)*C-(D-E))^(F+G) \rightarrow ?$
- ▶ $A-B/(C*D^E) \rightarrow ?$

Procedure for Conversion of Infix to Prefix using Stack

- Reverse the **infix** expression given in the problem.
- Scan the expression from left to right.
- Whenever the operands arrive, print them.
- If operator arrives and stack is empty, then **Push** the operator into the stack.
- If operator has higher or same precedence than **TOP** of stack, **Push** operator into the stack.
- If operator has lower precedence than **TOP** of stack, **Pop** and print **Top** of stack
- When end of expression is reached, **Pop** and print all operators from **TOP** of stack.
- If operator is ')', then **Push** it into the stack.
- If operator is '(', then **Pop** all operators from stack till it finds '(' in the stack.
- If the top of the stack is ')', **Push** the operator into the stack.
- At the end, **Reverse** the output.

Algorithm of Infix to Prefix

- Step 1.** Push “)” onto STACK, and add “(“ to end of the A
- Step 2.** Scan A from right to left and repeat step 3 to 6 for each element of A until the STACK is empty
- Step 3.** If an operand is encountered, add it to B
- Step 4.** If a right parenthesis is encountered, push it onto STACK
- Step 5.** If an operator is encountered then:
- Repeatedly pop from STACK and add to B each operator (on the top of STACK) which has higher precedence than the operator.
 - Add operator to STACK
- Step 6.** If left parenthesis is encountered then
- Repeatedly pop from STACK and add to B (each operator on top of stack until a right parenthesis is encountered)
 - Remove the left & right parenthesis
- Step 7.** Exit

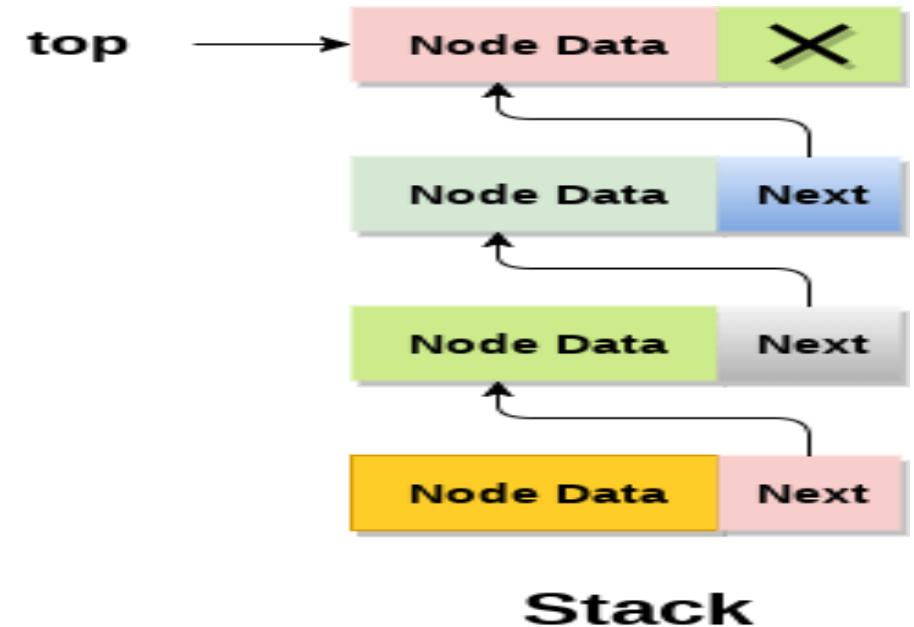
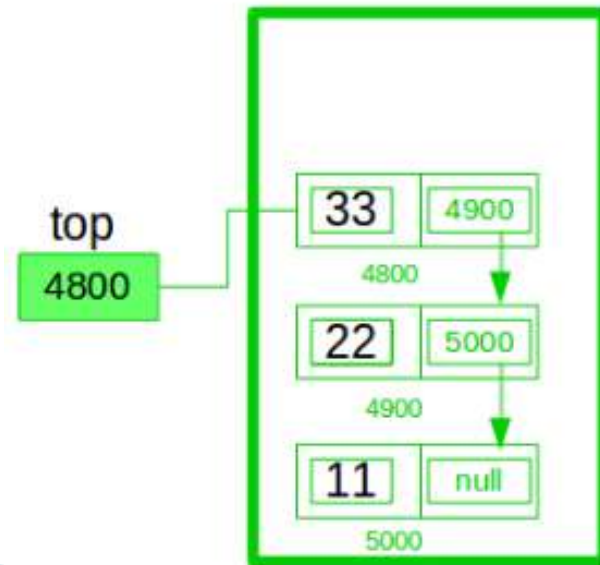


- $K + L - M * N + (O^P) * W/U/V * T + Q$
- To convert the expression from infix to prefix, first reverse the expression.
- The Reverse expression would be:
 $Q + T * V/U/W *) P^O(+ N*M - L + K$

Input expression	Stack	Prefix expression
Q		Q
+	+	Q
T	+	QT
*	+	QT
V	+	QTV
/	+	QTV
U	+	QTVU
/	+	QTVU
W	+	QTVUW
*	+	QTVUW
)	+	QTVUW
P	+	QTVUWP
^	+	QTVUWP
O	+	QTVUWPO
(	+	QTVUWPO^
+	++	QTVUWPO^*
N	++	QTVUWPO^*N
*	++	QTVUWPO^*N
M	++	QTVUWPO^*NM
-	++	QTVUWPO^*NM*
L	++	QTVUWPO^*NM*L
+	++	QTVUWPO^*NM*L
K	++	QTVUWPO^*NM*LK
		QTVUWPO^*NM*LK+-++
	++-+KL*MN*/*^OPWUVTQ	

Linked list implementation of stack

- Instead of using array, linked list can also be used to implement stack.
- Linked list allocates the memory dynamically.
- Time complexity in both the scenario is same for all the operations i.e. push, pop and peek.
- In linked list implementation of stack, the nodes are maintained non-contiguously in the memory.
- Each node contains a pointer to its immediate successor node in the stack.
- Stack is said to be overflow if the space left in the memory heap is not enough to create a node.
- The top most node in the stack always contains null in its address field.



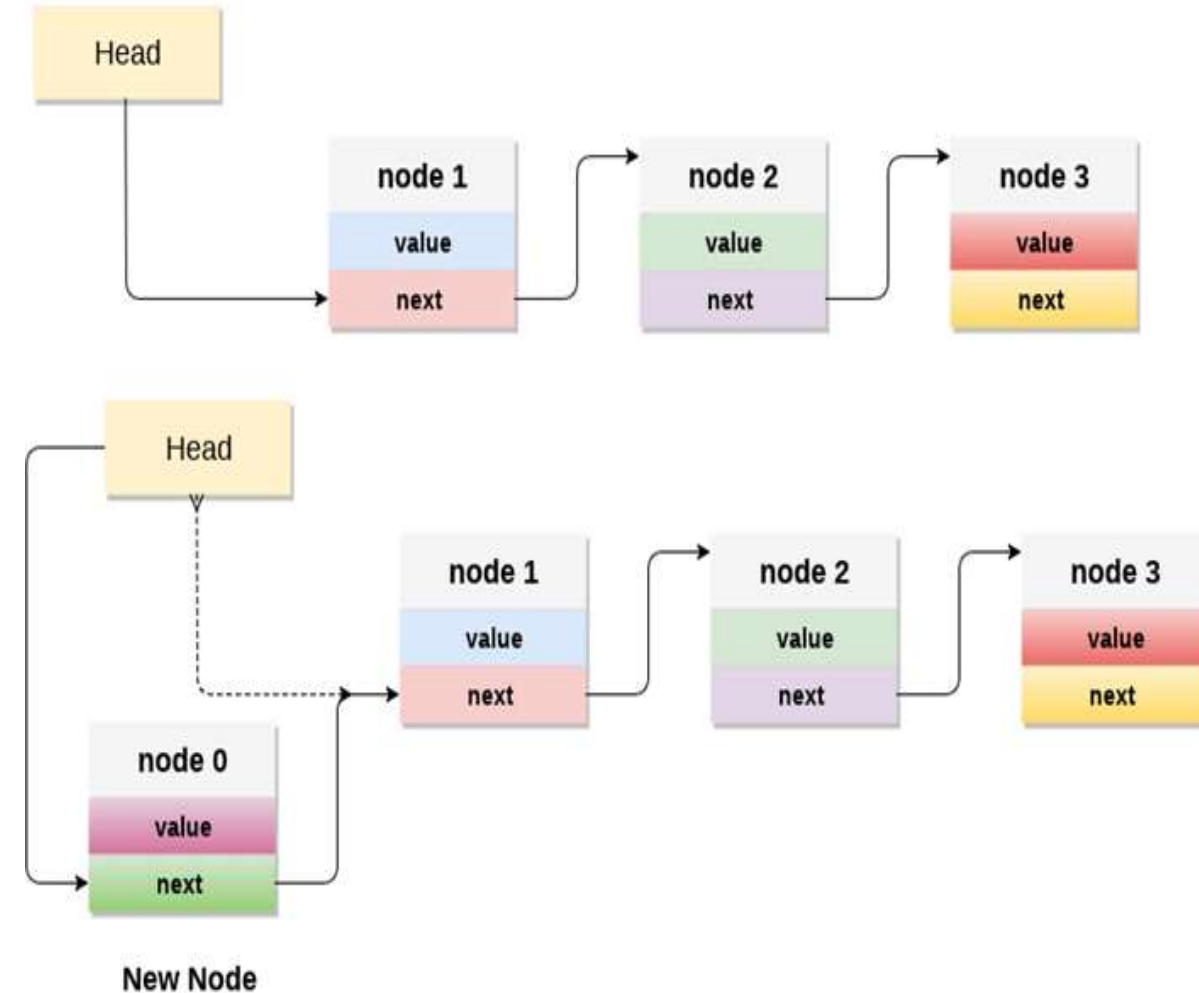
Adding a node to the stack (Push operation)

Pushing an element to a stack in linked list implementation is different from that of an array implementation.

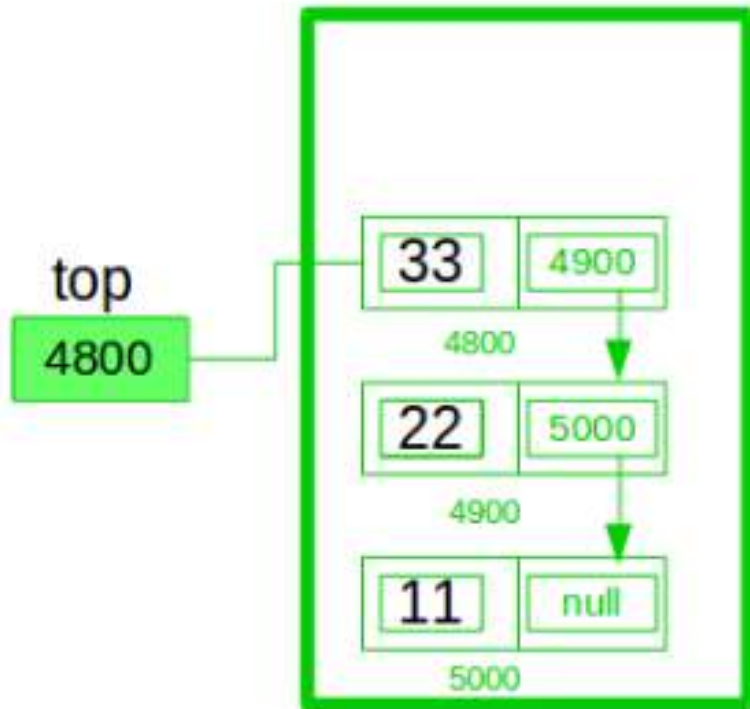
Procedure to push an element onto the stack:-

- Create a node first and allocate memory to it.
- If the list is empty then the item is to be pushed as the start node of the list.
- Assign value to the data part of the node and null to the address part of the node.
- If there are some nodes in the list already, then add new element in the beginning of the list.
- Assign the address of the starting element to the address field of the new node
- Make the new node as the starting node of the list.

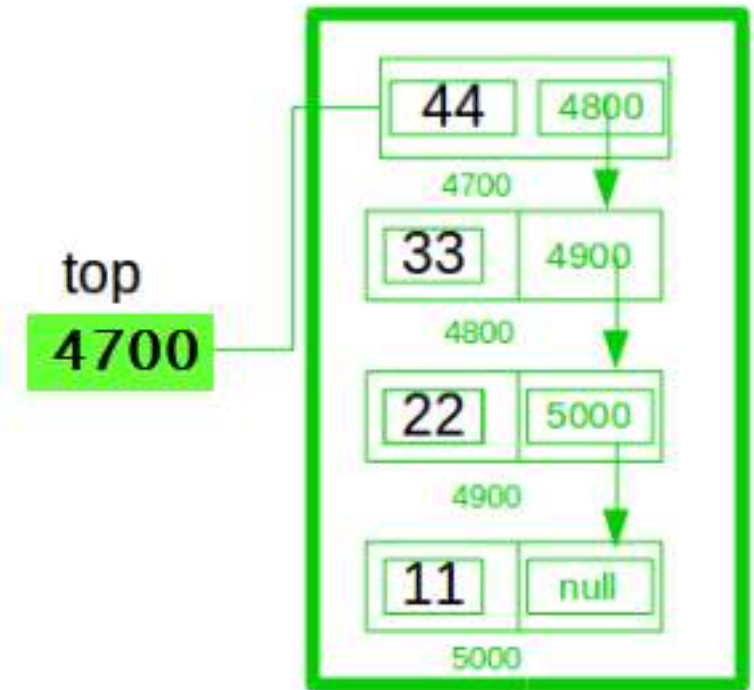
Time Complexity : $O(1)$



Push operation



Initial Stack Having Three element
And top have address 4800



First create a temp node and assign 44
Into data field and top in link field
And in last assign temp assign into top

Deleting a node from the stack (POP operation)

Deleting a node from the top of stack is referred to as **pop** operation.

Deleting a node from linked list implementation of stack is different from that in array implementation.

To POP an element from the stack, steps are:

Check for the underflow condition:

The underflow condition occurs when we try to pop from an already empty stack.

The stack will be empty if the head pointer of the list points to null.

Adjust the head pointer accordingly:

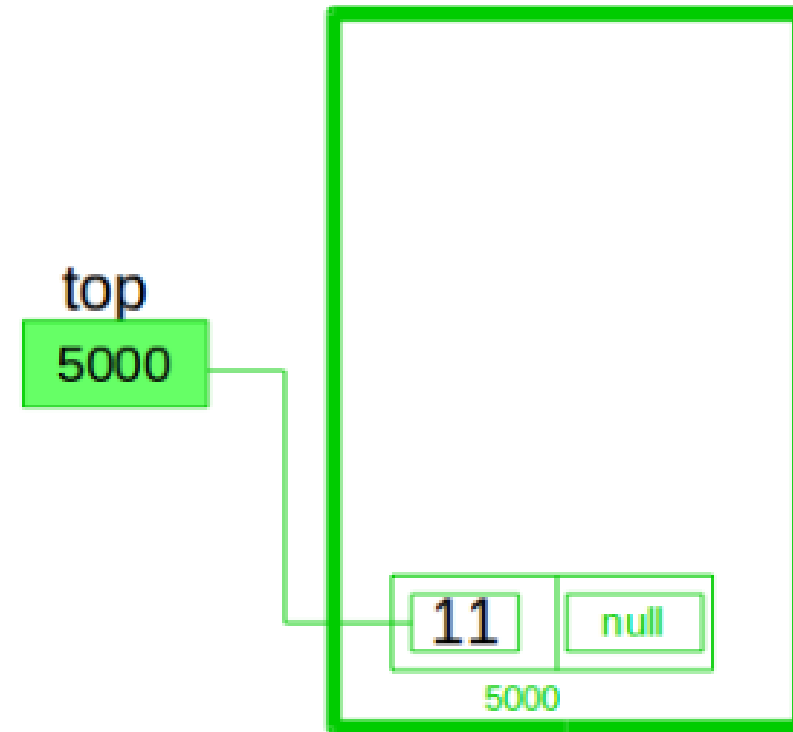
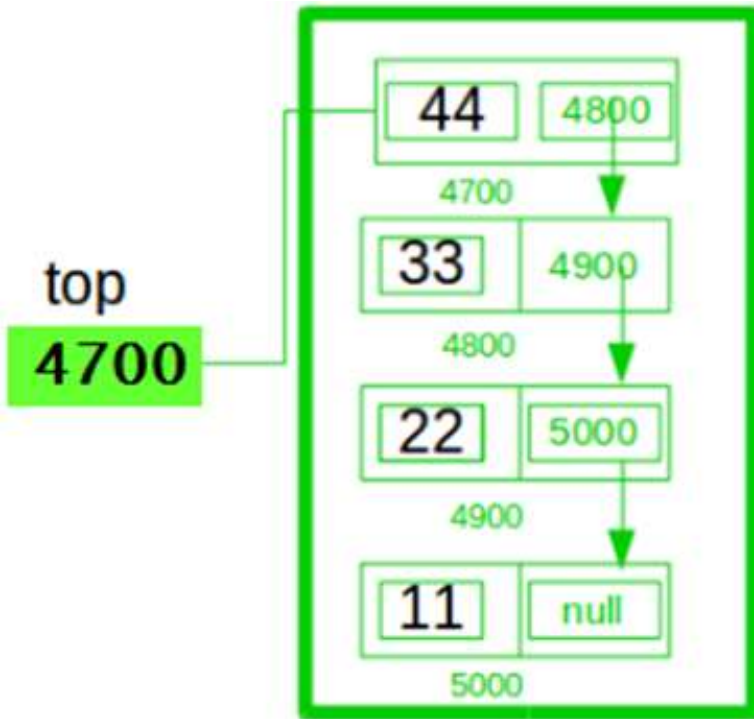
In stack, the elements are popped only from one end

The value stored in the head pointer must be deleted and the node must be freed.

The next node of the head node now becomes the head node.

Time Complexity : $O(n)$

POP operation



```
Pop three element  
temp = top;  
top = top->link;  
temp->link = NULL;  
free(temp);
```

Display the nodes (Traversing)

Displaying all the nodes of a stack needs traversing all the nodes of the linked list organized in the form of stack.

Steps for display operation:-

1. Copy the head pointer into a temporary pointer.
2. Move the temporary pointer through all the nodes of the list
3. Print the value field attached to every node.

Time Complexity : $O(n)$

C implementation

```
void display()
{ int i;
  struct node *ptr;
  ptr=head;
  if(ptr == NULL)
  { printf("Stack is empty\n"); }
  else
  { printf("Printing Stack elements \n");
    while(ptr!=NULL)
    { printf("%d\n",ptr->val);
      ptr = ptr->next;
    } }
}
```

THANK YOU